Pandit Deendayal Energy University
Department of Computer Science and Engineering

Cloud Architecture and Services

Assignment: Design a Prototype of a Containerized Cloud-Based Application

Project Title:

TravelNest

A Tourism and Travel Planning System

Submitted By:
Tapan Patel | Prem Patel | Milav Patel | Hozefa Travadi

Submission Date: April 11, 2026
Maximum Marks: 15 | Course Outcome: CO6

1. Problem Identification

Travel planning in the modern era involves a complex web of tasks that are typically scattered across multiple platforms. Travelers must simultaneously manage flight bookings on airline portals, hotel accommodations on hospitality websites, local transport arrangements through separate applications, and travel document storage via email or physical files. This fragmented ecosystem leads to significant inefficiencies and increases the chances of missed bookings, forgotten itinerary details, and budget overruns.

The tourism and travel industry generates billions of transactions annually and involves users across diverse demographics, ranging from solo backpackers to corporate travel managers. Despite the ubiquity of travel applications, no single platform effectively consolidates the full lifecycle of a trip — from initial planning and destination discovery to real-time budget tracking, AI-powered recommendations, and post-trip reviews — into a unified, accessible, and intelligent system.

Moreover, conventional travel management applications are often built as monolithic systems that are difficult to scale during peak travel seasons such as summer holidays and long weekends, when user demand can surge by several hundred percent. These legacy systems fail to leverage the elasticity of modern cloud infrastructure, resulting in downtime, slow response times, and poor user experiences precisely when demand is at its highest.

TravelNest is proposed as a solution to address these challenges. It is a cloud-native, containerized tourism and travel planning system designed to aggregate all aspects of trip management into a single, intelligent platform. By leveraging Docker containers for portability, Amazon Web Services (AWS) for cloud hosting and services, and an AI-driven trip planning module for intelligent recommendations, TravelNest aims to deliver a seamless and scalable digital experience to travellers worldwide.

2. Application Overview

TravelNest is a full-stack, cloud-based Tourism and Travel Planning System designed to provide travelers with a comprehensive digital companion for every stage of their journey. The application unifies trip planning, booking management, budget tracking, document storage, AI-powered recommendations, and community reviews within a single cohesive platform.

2.1 Main Users of the System

The system is designed to serve three primary categories of users:

- Registered Travelers: Individual users who create accounts to plan trips, manage bookings, track budgets, upload travel documents, and interact with the AI travel planner.
- Travel Administrators: Platform administrators who manage destination listings, moderate reviews, oversee user activity, and ensure content accuracy through a dedicated admin panel.
- Guest Users: Unregistered visitors who may browse destination listings and explore platform features before creating an account.

2.2 Key Functionalities

TravelNest provides the following eight core functional modules, each addressing a specific pain point in the travel planning lifecycle:

- Trip Planning: Users can create and manage detailed trip itineraries, including destination selection, travel dates, group size, and travel style preferences. The module supports PLANNING, ACTIVE, and COMPLETED trip statuses.
- Destination Discovery: A curated database of global destinations with filtering by continent, country, travel season, budget range, and tags. Each destination includes ratings, average cost per day, and geo-coordinates for map integration.

- Booking Management: Users can record and track flight, hotel, and activity bookings associated with specific trips. Booking statuses including PENDING, CONFIRMED, and CANCELLED are managed centrally.
- Budget Tracker: A dedicated financial tracking module that allows users to define total budgets per trip and log individual expenses by category.
- Document Vault: A secure repository for uploading and retrieving travel documents such as passports, visas, tickets, and insurance certificates. Documents are stored securely in AWS S3 with controlled access.
- Reviews and Ratings: A community-driven review system where users can post ratings and feedback for destinations they have visited, enriching the discovery experience for other travellers.
- AI Travel Planner: An intelligent module that accepts natural language queries from users and returns personalized trip recommendations, itinerary suggestions, and destination insights based on user preferences and travel history.

3. Frontend Layer

The frontend of TravelNest is a modern, responsive Single Page Application (SPA) built using React with TypeScript. It serves as the primary interface through which users interact with all platform features. The frontend is designed with a component-driven architecture, enabling modular development, high reusability, and maintainability.

3.1 Technology and Framework

The following technologies constitute the frontend technology stack:

- React: A declarative JavaScript library used for building the user interface through composable, reusable components. React's virtual DOM ensures efficient rendering and smooth interactivity.
- TypeScript: A statically typed superset of JavaScript that enhances code quality, provides compile-time error detection, and improves developer productivity through intelligent tooling.
- Tailwind CSS: A utility-first CSS framework that enables rapid and consistent UI styling without writing custom CSS.
- Vite: A next-generation frontend build tool that provides near-instantaneous hot module replacement (HMR) during development and highly optimized production builds.

3.2 Application Pages and Features

The key pages for Frontend are described below:

- Landing Page: Features a hero section, destination highlights, platform features, testimonials, pricing plans, and a call-to-action banner. This is the public-facing entry point designed to convert visitors into registered users.
- Authentication Pages: Dedicated login and signup pages that communicate with the backend authentication API using JWT tokens. Google OAuth integration is also supported for streamlined onboarding.

- User Dashboard: A centralized control panel providing a summary of active trips, upcoming bookings, recent expenses, and quick navigation to all modules.
- Trip Planner: An interactive interface for creating and managing trips, including itinerary timelines, activity scheduling, and trip status updates.
- Destination Discovery: A searchable and filterable grid of global destinations with rich detail pages including maps, average costs, best seasons, and community reviews.
- Booking Management: A table-based interface for viewing, adding, and updating booking records associated with each trip.
- Document Vault: A file management interface for uploading, categorizing, and retrieving travel documents stored in AWS S3.
- AI Trip Planner: A conversational interface where users can type natural language queries to receive AI-generated travel recommendations.

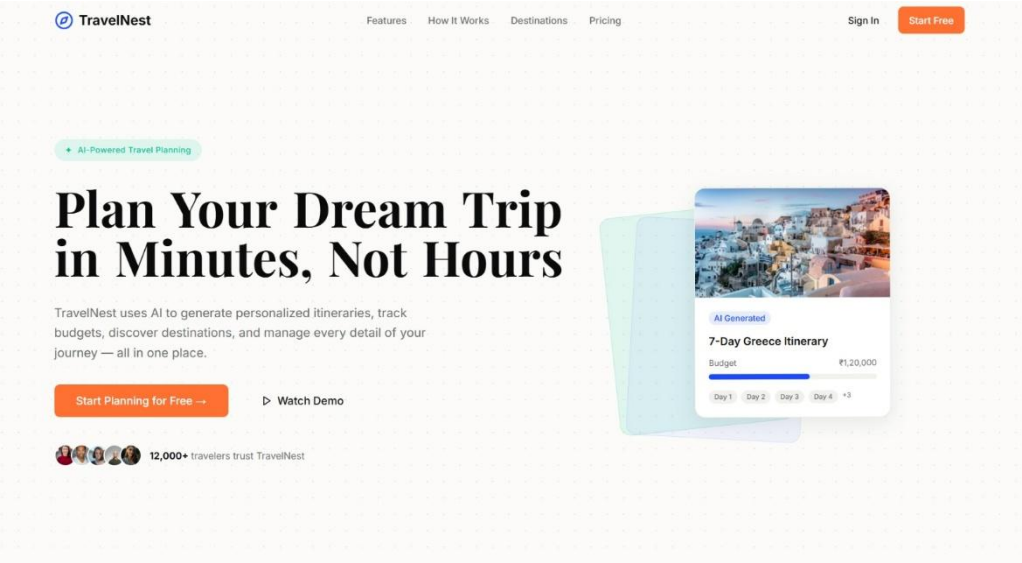


Figure 1: TravelNest Landing Page - Hero Section

This screenshot displays the TravelNest landing page hero section, showcasing the platform's primary value proposition with a visually compelling background, headline text, and call-to-action buttons for user registration and destination exploration.

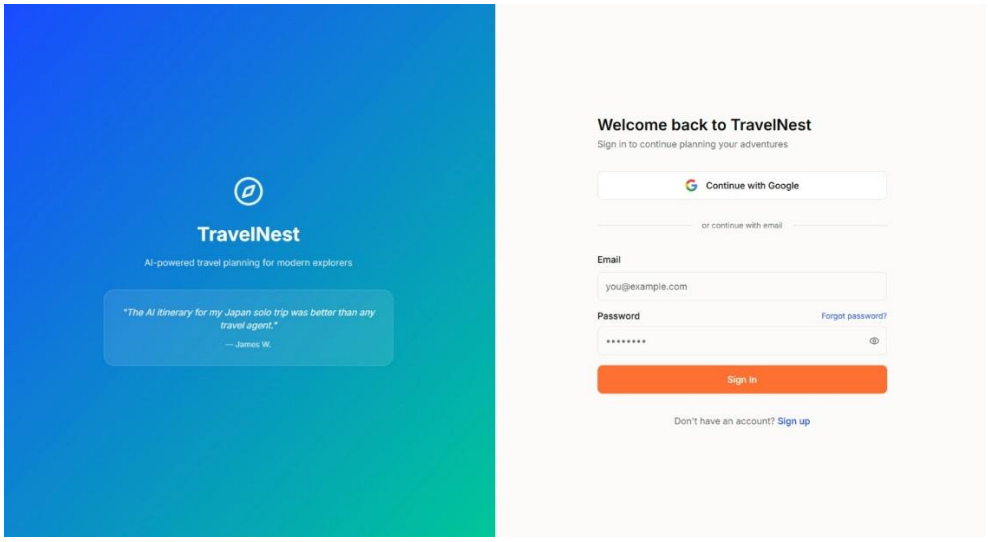


Figure 2: TravelNest Login Page Interface

This screenshot illustrates the TravelNest authentication page, displaying the login form with email and password fields, Google OAuth sign-in option, and form validation feedback for secure user authentication.

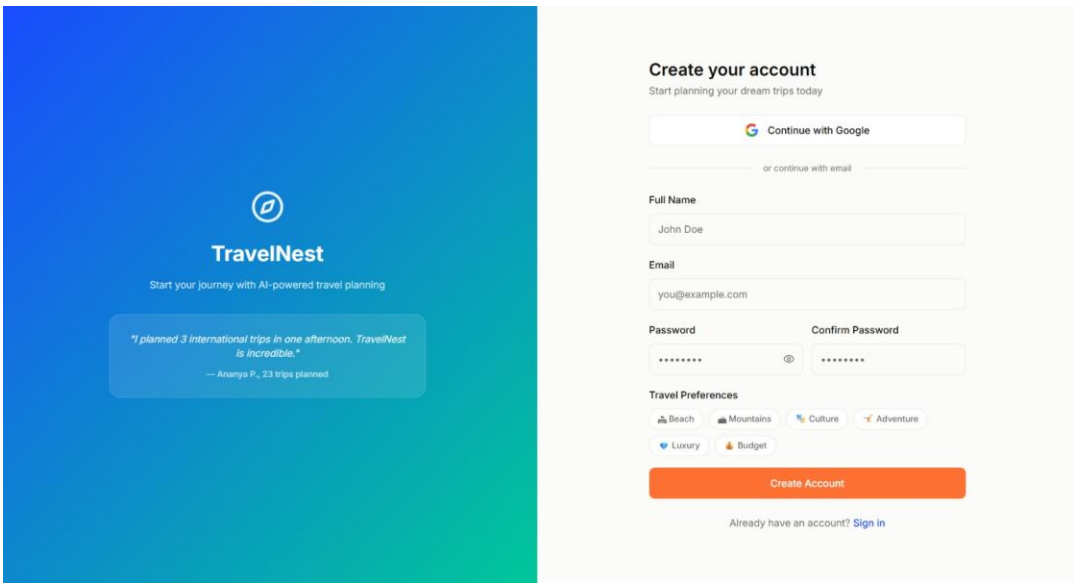


Figure 3: TravelNest Signup Page

This screenshot shows the user registration page where new travellers can create a TravelNest account by providing their name, email address, password, and optional profile preferences to personalize their experience.

4. Backend Layer

The backend of TravelNest is a RESTful API server built using Node.js with TypeScript and the Express framework. It is responsible for processing all client requests, enforcing business logic, managing database interactions, integrating with external cloud services, and coordinating with the AI service container.

4.1 Technology and Framework

- Node.js: A server-side JavaScript runtime environment built on Chrome's V8 engine. Its non-blocking, event-driven I/O model makes it well-suited for handling concurrent API requests from multiple users.
- Express: A minimal and flexible Node.js web application framework that provides robust middleware support, routing capabilities, and easy integration with third-party libraries.
- TypeScript: Applied consistently in the backend to enforce type safety, reduce runtime errors, and improve API contract clarity between modules.
- Prisma ORM: An open-source, type-safe database client for Node.js and TypeScript. Prisma handles all database schema definition, migrations, and query generation for the PostgreSQL database.
- PostgreSQL: The primary relational database system used by TravelNest, storing structured data including user profiles, trips, destinations, bookings, budgets, and reviews.
- MongoDB: A NoSQL document database used alongside PostgreSQL for storing flexible, schema-less data such as itinerary activity logs, which may vary significantly in structure.
- Redis: An in-memory data store used for caching frequently accessed data such as destination listings and session metadata, significantly reducing database query load and improving response times.

4.2 Module Architecture

The backend is organized using a modular architecture where each business domain is encapsulated in its own module directory containing a controller, service, routes, and schema file. The following modules are implemented:

- Auth Module: Handles user registration, login, JWT token generation, refresh token management, and Google OAuth passport integration.
- Users Module: Manages user profile retrieval and updates, including travel style preferences and profile avatar storage via AWS S3.
- Trips Module: Provides CRUD operations for trip creation, status updates, and retrieval by user.
- Destinations Module: Manages the global destination catalog with filtering and search capabilities.
- Bookings Module: Handles booking record creation, status transitions, and association with specific trips.
- Budget Module: Tracks budget entries, categorizes expenses, and computes spending summaries per trip.
- Documents Module: Manages secure file uploads to AWS S3 and maintains document metadata in the database.
- Alerts Module: Creates and manages user notification records, integrated with AWS SQS for scheduled message delivery.
- Reviews Module: Handles destination review submission, retrieval, and aggregate rating computation.
- AI Module: Provides a proxy endpoint to the Python-based AI service container, forwarding user queries and returning intelligent travel recommendations.

4.3 Middleware and Security

The backend implements several middleware layers to ensure security, consistency, and robustness. These include JWT authentication middleware that validates Bearer tokens on protected routes, role-based access control middleware that restricts admin operations to authorized users, request validation middleware using Zod schemas, rate limiting middleware to prevent API abuse, and centralized error handling middleware that returns structured error responses.

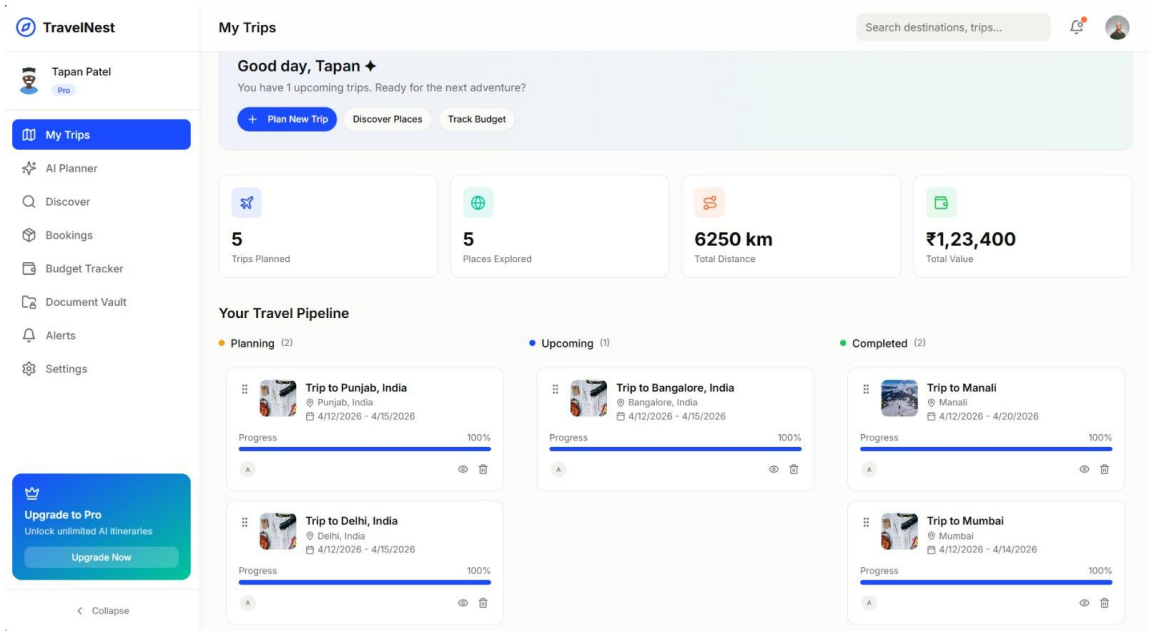


Figure 4: User Dashboard Interface

This screenshot presents the TravelNest user dashboard, which serves as the central hub displaying active trip summaries, upcoming booking reminders, recent budget entries, and quick navigation links to all major application modules.

```
Problems 10   Output   Debug Console   Terminal   Ports

PS C:\Users\Tapan\Desktop\TRIP> cd .\backend\
PS C:\Users\Tapan\Desktop\TRIP\backend> npm run dev

 > travelnest-backend@1.0.0 dev
 > nodemon --exec ts-node src/server.ts

 [nodemon] 3.1.14
 [nodemon] to restart at any time, enter `rs`
 [nodemon] watching path(s): *.*
 [nodemon] watching extensions: ts,json
 [nodemon] starting `ts-node src/server.ts`
 Redis Connected
 MongoDB Connected
 Server running in development mode on port 5000
 API Docs available at http://localhost:5000/api-docs
```

Figure 5: Backend API Running in Terminal

This screenshot captures the Node.js backend server output in the terminal, confirming successful startup of the Express API on port 5000, along with database connection confirmations for PostgreSQL, MongoDB, and Redis services.

```
TRIP
  ai-service
    __pycache__
    agent
      __pycache__
      __init__.py
      ai_engine.py
      api_services.py
      data_layer.py
      fallback_engine.py
      input_parser.py
      orchestrator.py
      travel_planner.py
      validator.py
    venv
    .env
    Dockerfile
    logger.py
    main.py
    requirements.txt
  backend
  frontend
  lambda\notification-service
    node_modules
    notification-service.zip
    notificationHandler.js
    package-lock.json
    package.json
  nginx
    nginx.conf
  .env
  .gitignore
  docker-compose.yml
  README.md
```

Figure 6: Project Folder Structure

5. Containerization Using Docker

Docker is the containerization platform used by TravelNest to package, distribute, and run all application services in a consistent, portable, and isolated manner. Each component of the TravelNest system is encapsulated within its own Docker container, enabling uniform deployment across development, testing, and production environments.

5.1 Frontend Container

The frontend service is containerized using a multi-stage Dockerfile that first builds the React application using the Node.js base image and then serves the compiled static files using the lightweight Nginx stable-alpine image. This approach significantly reduces the final image size by excluding development dependencies and build tooling from the production image. The frontend container exposes port 80 and is managed behind the Nginx reverse proxy.

5.2 Backend Container

The backend API is containerized using a multi-stage Dockerfile employing the Node.js 20 Alpine base image. In the builder stage, all Node.js dependencies are installed, the Prisma client is generated, and the TypeScript source code is compiled to JavaScript. The production stage copies only the compiled output, node_modules, and Prisma schema files, then installs PM2 as a process manager to ensure the Node.js process restarts automatically in case of failure. The backend container exposes port 5000.

5.3 AI Service Container

The AI service is a Python-based microservice packaged in its own Docker container. It provides a dedicated endpoint for processing AI travel planning queries. The service communicates with the backend container over the internal Docker network and is also accessible through the Nginx proxy.

5.4 Supporting Service Containers

In addition to the application containers, the Docker Compose configuration defines containers for the following supporting services:

- PostgreSQL 15 Alpine: The primary relational database container with health check configuration to ensure the backend waits for database readiness before starting.
- MongoDB 7: The NoSQL database container used for itinerary and flexible document storage.
- Redis 7 Alpine: The in-memory caching container used to improve response times for frequently accessed data.
- Nginx Alpine: The reverse proxy container that routes incoming HTTP requests to the appropriate frontend, backend, or AI service container based on URL path rules.

5.5 Docker Compose Orchestration

All services are orchestrated using Docker Compose, which defines the complete multi-container application stack in a single docker-compose.yml file. Service dependencies are declared explicitly — the backend waits for both PostgreSQL and MongoDB health checks before starting, ensuring database readiness. All containers are configured with the restart: unless-stopped policy for high availability. Named volumes are used to persist PostgreSQL and MongoDB data across container restarts, ensuring data durability.

5.6 Advantages of Containerization

The adoption of Docker containerization in TravelNest provides several significant advantages:

- Portability: Docker containers encapsulate the application and all its dependencies, ensuring that TravelNest runs identically across any environment — developer laptops, CI/CD pipelines, or cloud virtual machines — without the 'it works on my machine' problem.
- Consistent Deployment: The same Docker images used during development are deployed to production, eliminating configuration drift and environment-specific bugs.
- Simplified Application Management: Docker Compose allows the entire TravelNest stack — including six distinct services — to be started, stopped, and managed with a single command, dramatically simplifying operational procedures.

- Isolation: Each service runs in its own isolated container with its own filesystem and network namespace, preventing dependency conflicts between services.
- Scalability: Individual containers can be independently scaled horizontally by running multiple replicas behind the Nginx load balancer, enabling the system to handle increased traffic without redeploying the entire stack.

Dockerfile

frontend > Dockerfile > ...

```
# Build stage
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM nginx:stable-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Ctrl+L for Agent

Figure 7: Dockerfile for Frontend Service

This screenshot displays the multi-stage Dockerfile for the TravelNest frontend container, showing the Node.js build stage that compiles the React application and the Nginx production stage that serves the compiled static assets.

```
Dockerfile ×
backend > Dockerfile > ...
1   FROM node:20-alpine AS builder
2   WORKDIR /app
3   COPY package*.json ./
4   RUN npm ci
5   COPY . .
6   RUN npm run prisma:generate
7   RUN npm run build
8
9   FROM node:20-alpine AS production
10  RUN apk add --no-cache openssl libc6-compat
11  WORKDIR /app
12  COPY --from=builder /app/dist ./dist
13  COPY --from=builder /app/node_modules ./node_modules
14  COPY --from=builder /app/package.json ./
15  COPY --from=builder /app/src/prisma ./src/prisma
16  RUN npm install -g pm2
17  EXPOSE 5000
18  COPY start.sh ./
19  RUN chmod +x start.sh
20  CMD ["sh", "start.sh"]
21
```

Figure 8: Dockerfile for Backend Service

This screenshot shows the multi-stage Dockerfile for the TravelNest backend container, illustrating the builder stage that installs dependencies and compiles TypeScript, and the production stage that runs the compiled application using PM2 process manager.

trip	-	-	-	78.52%	2 seconds ago
nginx-1	afd36af43f89	nginx:alpine	80:80	0%	2 seconds ago
frontend-1	dbdadfc97f98	trip-frontend		0%	4 seconds ago
backend-1	97a797c42a05	trip-backend		76.81%	5 seconds ago
redis-1	4ccfc118e4fa	redis:7-alpine	6379:6379	0.56%	11 minutes ago
postgres-1	ec334310fe46	postgres:15-alpine	5432:5432	0.32%	11 minutes ago
mongodb-1	5c6dfcfe9626	mongo:7	27017:27017	0.69%	11 minutes ago
ai-service-1	c9fa7e9f89df	trip-ai-service		0.14%	9 seconds ago

Figure 9: Docker Compose Configuration File

This screenshot presents the complete docker-compose.yml file for TravelNest, displaying all six defined services — nginx, backend, frontend, ai-service, postgres, mongodb, and redis — along with their environment variables, health checks, dependencies, and volume configurations.

```
PS C:\Users\Tapan> docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS                    PORTS
          NAMES
afd36af43f89   nginx:alpine        "/docker-entrypoint.…"   18 hours ago   Up 35 minutes             0.0.0.0:80->80/tcp, [::]:80->80/tcp
          trip-nginx-1
dbdadfc97f98   trip-frontend       "/docker-entrypoint.…"   18 hours ago   Up 35 minutes             80/tcp
          trip-frontend-1
97a797c42a05   trip-backend        "docker-entrypoint.s…"   18 hours ago   Up 35 minutes (unhealthy)   5000/tcp
          trip-backend-1
4ccfc118e4fa   redis:7-alpine      "docker-entrypoint.s…"   18 hours ago   Up 45 minutes             0.0.0.0:6379->6379/tcp, [::]:6379->6379/
tcp       trip-redis-1
ec334310fe46   postgres:15-alpine  "docker-entrypoint.s…"   18 hours ago   Up 45 minutes (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/
tcp       trip-postgres-1
5c6dfcfe9626   mongo:7             "docker-entrypoint.s…"   18 hours ago   Up 45 minutes             0.0.0.0:27017->27017/tcp, [::]:27017->27
017/tcp   trip-mongodb-1
c9fa7e9f89df   trip-ai-service     "uvicorn main:app --…"   18 hours ago   Up 35 minutes (unhealthy)   8000/tcp
          trip-ai-service-1
PS C:\Users\Tapan>
```

Figure 10: Docker Containers Running

This screenshot captures the output of the docker ps command showing all TravelNest containers in a running state, including their container IDs, image names, exposed ports, uptime durations, and current health status.

```
Containers / trip-redis-1

trip-redis-1
4ccfc118e4fa   redis:7-alpine
6379:6379

STATUS
Running (11 minutes ago)

Logs   Inspect   Bind mounts   Exec   Files   Stats

1:C 11 Apr 2026 18:28:51.051 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 11 Apr 2026 18:28:51.051 * Redis version=7.4.8, bits=64, commit=00000000, modified=0, pid=1, just started
1:C 11 Apr 2026 18:28:51.051 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server
/path/to/redis.conf
1:M 11 Apr 2026 18:28:51.052 * monotonic clock: POSIX clock_gettime
1:M 11 Apr 2026 18:28:51.053 * Running mode=standalone, port=6379.
1:M 11 Apr 2026 18:28:51.054 * Server initialized
1:M 11 Apr 2026 18:28:51.054 * Ready to accept connections tcp
1:C 11 Apr 2026 18:39:57.735 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 11 Apr 2026 18:39:57.736 * Redis version=7.4.8, bits=64, commit=00000000, modified=0, pid=1, just started
1:C 11 Apr 2026 18:39:57.736 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server
/path/to/redis.conf
1:M 11 Apr 2026 18:39:57.736 * monotonic clock: POSIX clock_gettime
1:M 11 Apr 2026 18:39:57.738 * Running mode=standalone, port=6379.
1:M 11 Apr 2026 18:39:57.739 * Server initialized
1:M 11 Apr 2026 18:39:57.740 * Ready to accept connections tcp
1:C 12 Apr 2026 04:33:09.328 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 12 Apr 2026 04:33:09.328 * Redis version=7.4.8, bits=64, commit=00000000, modified=0, pid=1, just started
1:C 12 Apr 2026 04:33:09.328 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server
/path/to/redis.conf
1:M 12 Apr 2026 04:33:09.329 * monotonic clock: POSIX clock_gettime
1:M 12 Apr 2026 04:33:09.332 * Running mode=standalone, port=6379.
1:M 12 Apr 2026 04:33:09.332 * Server initialized
1:M 12 Apr 2026 04:33:09.333 * Ready to accept connections tcp
1:M 12 Apr 2026 05:33:10.048 * 1 changes in 3600 seconds. Saving...
1:M 12 Apr 2026 05:33:10.050 * Background saving started by pid 21
21:C 12 Apr 2026 05:33:10.068 * DB saved on disk
21:C 12 Apr 2026 05:33:10.069 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
```

Figure 11: Redis Server Running

This screenshot shows the Redis 7 Alpine container running within the Docker environment, confirming successful startup of the in-memory caching service that TravelNest uses to cache frequently accessed destination data and user session information.

6. Cloud Infrastructure

TravelNest is designed to be deployed on Amazon Web Services (AWS), leveraging the AWS Free Tier wherever applicable to minimize operational costs during the prototype phase while maintaining production-quality architecture. AWS provides the compute, storage, messaging, and networking infrastructure required to host and operate the containerized application.

6.1 Compute: AWS EC2 (Free Tier)

The Docker Compose application stack is hosted on an AWS EC2 t2.micro instance, which is eligible for the AWS Free Tier (750 hours per month in the first 12 months). The EC2 instance runs Ubuntu Server as the operating system, with Docker Engine and Docker Compose installed. The Nginx container within the Docker Compose stack handles incoming HTTP traffic on port 80, routing requests to the appropriate service containers.

6.2 File Storage: AWS S3

Amazon S3 is used as the file storage backend for the TravelNest Document Vault and user profile avatars. Files are uploaded directly from the backend API using the AWS SDK. The S3 bucket is configured with appropriate IAM policies to ensure that only authenticated users with valid signed URLs can access their own documents. The AWS Free Tier provides 5 GB of standard S3 storage, 20,000 GET requests, and 2,000 PUT requests per month.

6.3 Serverless Functions: AWS Lambda

AWS Lambda functions are used for lightweight serverless operations that do not warrant dedicated server resources. Specific use cases in TravelNest include processing thumbnail generation for uploaded document images, triggering email notification pipelines, and executing scheduled data cleanup jobs. Lambda functions are invoked asynchronously from the backend and receive message payloads through AWS SQS. The AWS Free Tier provides one million free Lambda requests and 400,000 GB-seconds of compute time per month.

6.4 Message Queue: AWS SQS

Amazon Simple Queue Service (SQS) is integrated into the TravelNest alerts and reminders pipeline. When a user creates a new trip or a booking deadline approaches, the backend publishes a message to an SQS queue. A Lambda function subscribes to the queue and processes each message to trigger the appropriate notification. SQS decouples the message production from consumption, ensuring reliable delivery even under high load. The AWS Free Tier includes one million SQS requests per month.

6.5 Database Services

TravelNest uses two databases to manage its data storage requirements:

- PostgreSQL (Relational SQL): The primary data store for all structured application data including users, trips, destinations, bookings, budgets, documents, reviews, and authentication tokens. A relational database is chosen because the data has well-defined relationships and requires ACID transaction guarantees — for example, a booking must always be linked to a valid trip and user. In production, this can be hosted on AWS RDS PostgreSQL Free Tier (db.t3.micro, 20 GB storage).
- MongoDB (Non-relational NoSQL): Used for storing itinerary activity data, which is inherently schema-flexible — different types of activities such as flights, meals, and sightseeing tours may have entirely different attribute sets. MongoDB's document model accommodates this variability without requiring schema migrations.

6.6 Networking and Security

The EC2 instance is placed within an AWS Virtual Private Cloud (VPC) with security groups configured to allow inbound HTTP traffic on port 80 and SSH access on port 22 restricted to authorized IP addresses. The database containers are not exposed to public internet access and are accessible only from within the Docker internal network. HTTPS can be enabled by attaching an AWS Application Load Balancer with an AWS Certificate Manager SSL certificate, secured at no additional cost for ACM certificates.

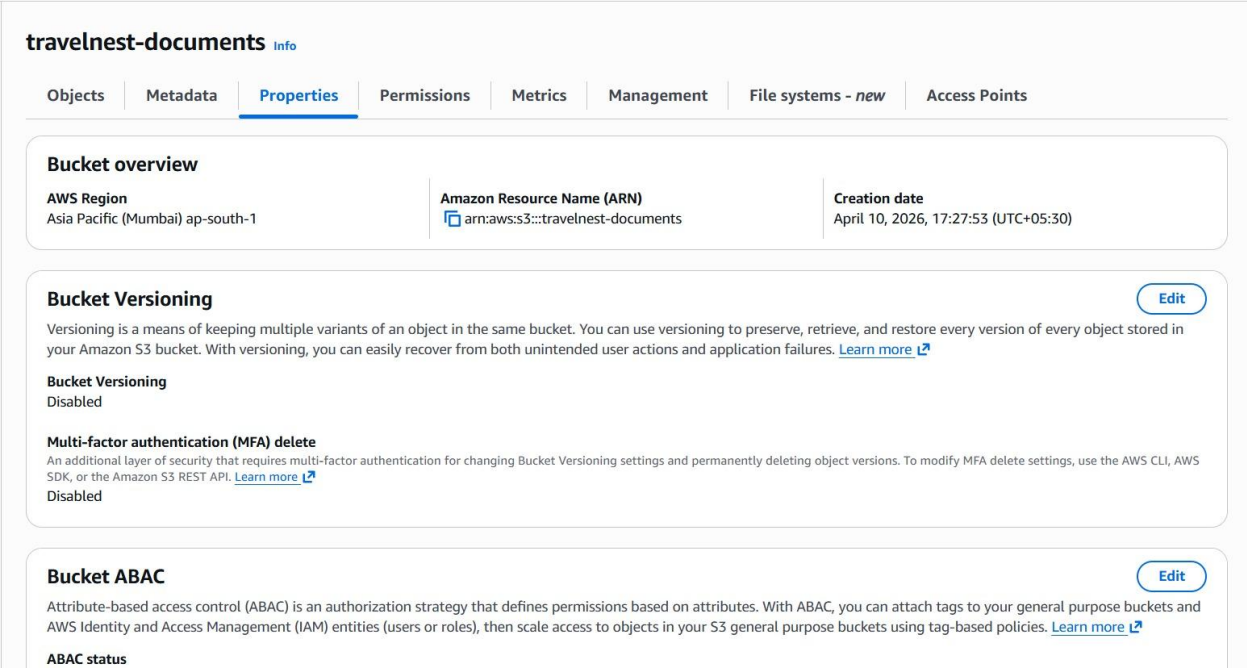
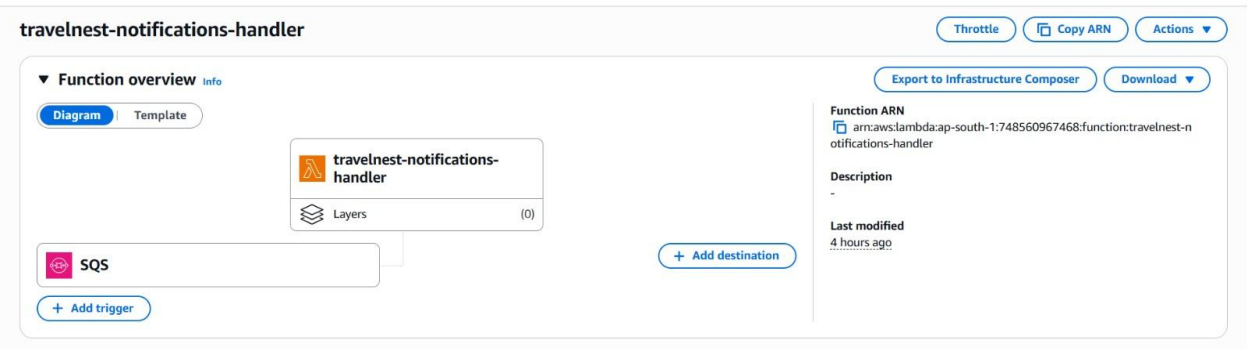


Figure 12: AWS S3 Bucket Configuration

This screenshot displays the AWS S3 management console showing the TravelNest storage bucket, including its name, region, access control settings, and the folder structure used to organize uploaded user documents and profile avatars.



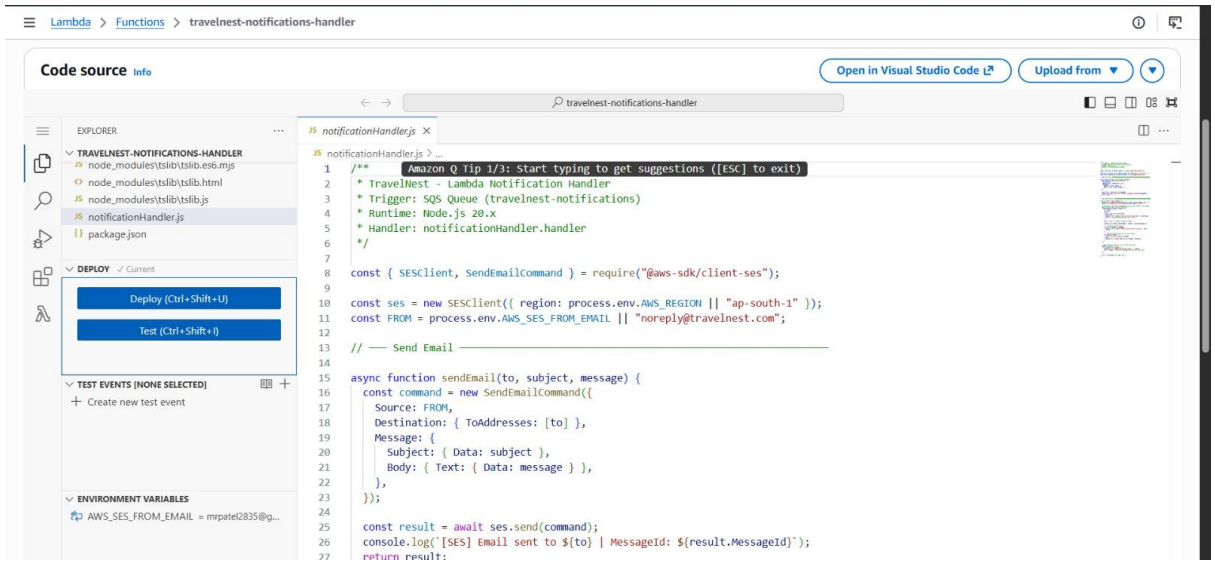


Figure 13: AWS Lambda Function

This screenshot shows the AWS Lambda console with the TravelNest notification processing function, displaying the function's runtime configuration, memory allocation, execution timeout, and the trigger configuration connecting it to the SQS message queue.

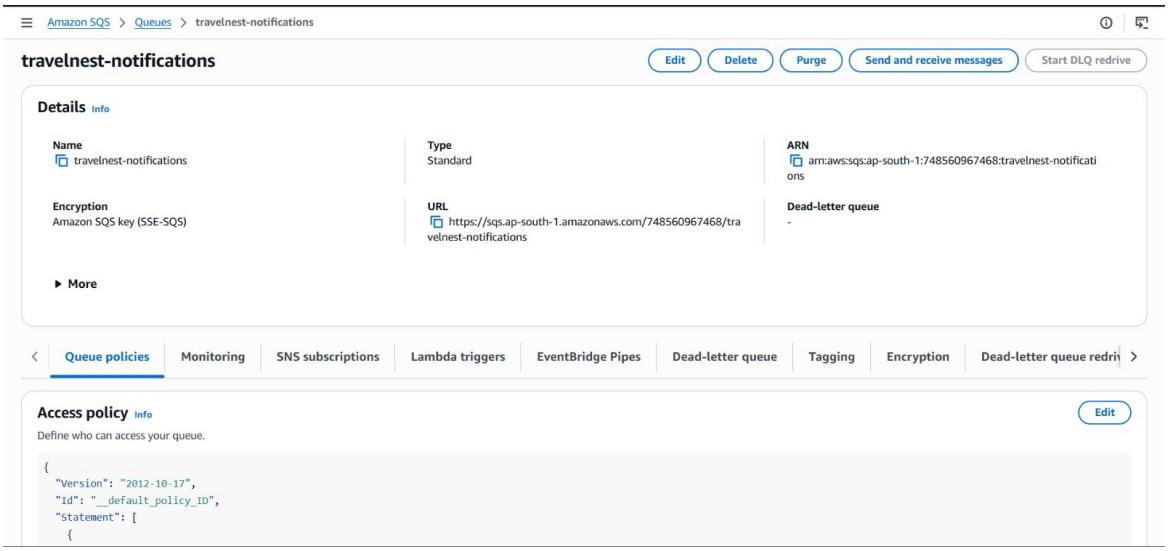


Figure 14: AWS SQS Queue

This screenshot illustrates the AWS SQS console showing the TravelNest alerts queue, including queue type (Standard), approximate number of messages, visibility timeout, and message retention period settings used for the trip reminder notification pipeline.

7. System Architecture Diagram

The TravelNest system architecture follows a layered, microservices-oriented design where each component serves a distinct role and communicates with adjacent components through well-defined interfaces. The architecture diagram was created using Draw.io and illustrates the complete flow from user interaction at the browser level through to cloud storage and serverless processing at the AWS infrastructure level.

The interaction between system components is described as follows:

- User: The end user accesses TravelNest via a standard web browser on a desktop or mobile device. All HTTP requests originate from the browser and are directed to the Nginx reverse proxy.
- Nginx Reverse Proxy Container: The Nginx container acts as the single entry point to the application. Based on the request URL path, it routes traffic to either the Frontend container (for page requests) or the Backend API container (for /api/* requests) or the AI Service container (for /ai/* requests).
- Frontend Container (React/Nginx): The frontend container serves the pre-built React SPA static assets. Once loaded in the user's browser, the React application makes AJAX calls directly to the backend API through the Nginx proxy.
- Backend Container (Node.js/Express): The backend API processes authenticated requests, executes business logic, performs database read and write operations, and invokes AWS services as required. It communicates with the PostgreSQL container for relational data, the MongoDB container for flexible document data, and the Redis container for caching.
- AI Service Container (Python): The AI service receives natural language travel queries from the backend and returns structured recommendations. It operates as an independent microservice with its own resource allocation.
- PostgreSQL Container: The primary relational data store for all structured entities within the TravelNest data model.
- MongoDB Container: The document database for flexible itinerary activity records.

- Redis Container: The in-memory cache for session data and frequently queried destination records.
- AWS Services (S3, Lambda, SQS): External cloud services integrated with the backend for file storage, serverless processing, and asynchronous message queuing respectively.

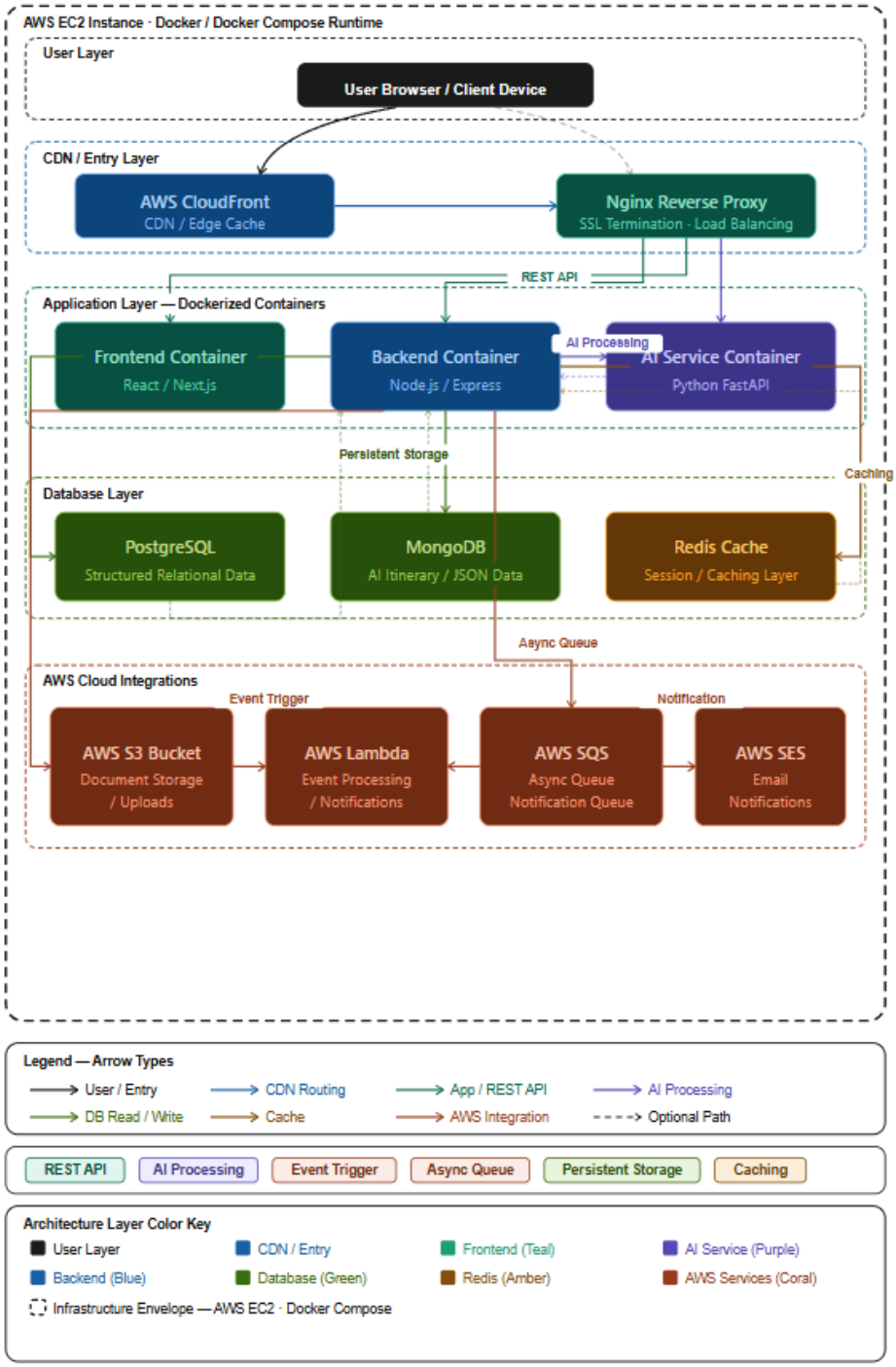


Figure 15: TravelNest Cloud Architecture Diagram

This diagram illustrates the complete TravelNest system architecture, showing the flow from user browser through Nginx reverse proxy to the frontend, backend, and AI service containers, connected to PostgreSQL, MongoDB, and Redis databases, with integrations to AWS S3, AWS Lambda, and AWS SQS cloud services.

8. Security Considerations

Security is a foundational concern in the design of TravelNest, given that the platform handles sensitive personal information including travel documents, financial data, and authentication credentials. The following security mechanisms are implemented across the system:

8.1 JWT Authentication

TravelNest implements JSON Web Token (JWT) based authentication for all protected API endpoints. Upon successful login, the backend issues a short-lived access token (signed with a secret key using the HS256 algorithm) and a long-lived refresh token stored in the database. The access token is included in the Authorization header of subsequent API requests as a Bearer token. The authentication middleware on the backend validates the token signature and expiry on every protected request, rejecting any request with an invalid, expired, or tampered token.

Refresh tokens enable users to obtain new access tokens without re-entering credentials, improving usability while limiting the risk exposure window of access tokens. Refresh tokens can be invalidated server-side, enabling forced logout functionality.

8.2 Secure API Design

All backend API endpoints are designed following the principle of least privilege. Route-level middleware enforces role-based access control, ensuring that standard users cannot access administrative operations such as destination management and user moderation. Input validation is enforced on all request payloads using Zod schemas to prevent injection attacks and malformed data from reaching the database layer. Rate limiting middleware restricts the number of requests per IP address per time window, mitigating brute force and denial-of-service attacks.

8.3 HTTPS Communication

All communication between the user's browser and the TravelNest server is encrypted using HTTPS (Hypertext Transfer Protocol Secure) with TLS (Transport Layer Security). In the cloud deployment, an AWS Application Load Balancer with an SSL certificate provisioned through AWS Certificate Manager (ACM) terminates TLS at the load balancer level and forwards

decrypted traffic to the EC2 instance over the private network. This ensures that all user data, including authentication credentials and document uploads, is transmitted in an encrypted form that cannot be intercepted by third parties.

8.4 Controlled Database Access

The PostgreSQL and MongoDB containers are not exposed on public network ports in the production deployment. They are accessible only from within the Docker internal network, meaning only the backend container can initiate database connections. Database credentials are stored as environment variables passed through Docker Compose environment files (.env) and are never hardcoded in source code. AWS S3 bucket policies are configured to deny all public access, with file retrieval possible only through pre-signed URLs generated by the authenticated backend API.

9. Scalability and Reliability

TravelNest is architected with scalability as a core design principle, enabling the system to accommodate growth in user base and transaction volume without requiring fundamental architectural changes.

9.1 Vertical Scaling

Vertical scaling refers to increasing the computational resources (CPU, RAM, storage) allocated to a single server instance. In the AWS deployment, this can be achieved by upgrading the EC2 instance type from the Free Tier t2.micro to larger instance types such as t3.medium, t3.large, or c5.xlarge, depending on observed resource utilization patterns. Vertical scaling is straightforward to implement since it requires no changes to the application architecture and can be performed with minimal downtime through AWS instance resize operations. However, vertical scaling has physical hardware limits and is generally considered a short-term scaling strategy.

9.2 Horizontal Scaling

Horizontal scaling refers to distributing load across multiple server instances running the same application. In the TravelNest architecture, this is achievable at the container level using AWS Elastic Container Service (ECS) or Kubernetes to run multiple replicas of the backend and AI service containers behind an AWS Application Load Balancer. Since each backend instance is stateless — session data is maintained in Redis and file state in S3 — requests can be routed to any available backend instance without session affinity concerns. Horizontal scaling allows the system to handle virtually unlimited concurrent users by adding instances proportionally to demand.

9.3 Docker Container Scaling

Docker Compose supports deploying multiple replicas of a service using the deploy.replicas configuration. In conjunction with Nginx's upstream load balancing, multiple backend container instances can share the request load within the same host machine. When migrating to Docker Swarm or Kubernetes, container scaling becomes fully automated based on CPU and memory utilization metrics, enabling elastic scaling without manual intervention.

9.4 Cloud Elasticity and Reliability

AWS infrastructure provides inherent elasticity through services such as AWS Auto Scaling Groups, which automatically add or remove EC2 instances based on predefined CloudWatch metrics such as average CPU utilization. AWS S3 provides virtually unlimited storage capacity with eleven nines (99.999999999%) durability. AWS SQS guarantees at-least-once message delivery, ensuring that trip reminder notifications are never lost even during backend downtime. The use of PostgreSQL with automated backup and point-in-time recovery (PITR) on AWS RDS further enhances data reliability and disaster recovery capabilities.

10. Advantages of Cloud and Container-Based Deployment

The decision to build TravelNest as a containerized, cloud-native application provides a comprehensive set of advantages over traditional monolithic, on-premise deployments:

- Cost Efficiency: By leveraging the AWS Free Tier for compute (EC2 t2.micro), storage (S3 5 GB), messaging (SQS 1 million requests), and serverless compute (Lambda 1 million requests), TravelNest can be operated at minimal cost during the initial growth phase.
- Global Availability: AWS operates data centers across multiple geographic regions and availability zones. As the user base grows globally, the application can be replicated across multiple regions using AWS CloudFront for content delivery.
- High Availability: AWS services such as EC2, RDS, and S3 offer Service Level Agreements (SLAs) and using docker resources guaranteeing 99.9% or higher uptime, the TravelNest application self-heals from container failures without manual intervention.
- Rapid Deployment: Docker containers enable the entire TravelNest application stack to be deployed on any Docker-enabled cloud instance within minutes using a single docker-compose up command. This accelerates development cycles and enables rapid iteration.
- Environment Consistency: Docker eliminates environment inconsistencies by packaging the application and its dependencies into immutable container images.
- Separation of Concerns: The microservices architecture, with separate containers for the frontend, backend, AI service, and databases, enables independent development, testing, deployment, and scaling of each component.
- Managed Cloud Services: By using AWS S3, Lambda, and SQS, TravelNest offloads the operational burden of managing file storage infrastructure, serverless function execution environments, and message queue systems to AWS, allowing the development team to focus on application logic.
- Security and Compliance: AWS provides a comprehensive security framework including IAM for access management, VPC for network isolation, ACM for SSL certificate management, and KMS for encryption key management, enabling TravelNest to implement enterprise-grade security without building security infrastructure from scratch.

11. Technology Stack Summary

The following table provides a comprehensive summary of all technologies and services employed in the TravelNest system, organized by architectural component:

Component	Technology / Service Used	Purpose
Frontend	React with TypeScript	User interface — Single Page Application
Styling	Tailwind CSS	Utility-first CSS framework for responsive design
Build Tool	Vite	Frontend build tooling and hot module replacement
Backend	Node.js with Express (TypeScript)	RESTful API server and business logic processing
ORM	Prisma ORM	Type-safe database schema management and queries
Containerization	Docker and Docker Compose	Packaging and orchestrating all application services
Reverse Proxy	Nginx Alpine	Request routing and load balancing between containers
Cloud Platform	Amazon Web Services (AWS)	Hosting infrastructure and cloud services
Compute	AWS EC2 t2.micro (Free Tier)	Running the Docker Compose application stack
Primary Database	PostgreSQL 15 (Relational SQL)	Structured data storage — users, trips, bookings
Secondary Database	MongoDB 7 (NoSQL)	Flexible document storage for itinerary activities
Cache	Redis 7 Alpine	In-memory caching for destinations and sessions

Component	Technology / Service Used	Purpose
File Storage	AWS S3 (Free Tier: 5 GB)	Secure storage for travel documents and avatars
Serverless	AWS Lambda (Free Tier)	Notification processing and scheduled data cleanup
Messaging	AWS SQS (Free Tier)	Asynchronous alert and reminder message queue
Authentication	JWT (JSON Web Tokens)	Stateless, secure user authentication
OAuth	Google OAuth via Passport.js	Social login for streamlined onboarding
AI Service	Python (containerized)	Natural language travel planning recommendations
Process Manager	PM2	Node.js process supervision in backend container
API Documentation	Swagger / OpenAPI	Interactive backend API documentation

12. Screenshots of the Completed Work with Description

The following figures present screenshots of the key interfaces, development environment configurations, and cloud service setups that constitute the completed TravelNest prototype. Each figure is accompanied by a description of its contents and significance.

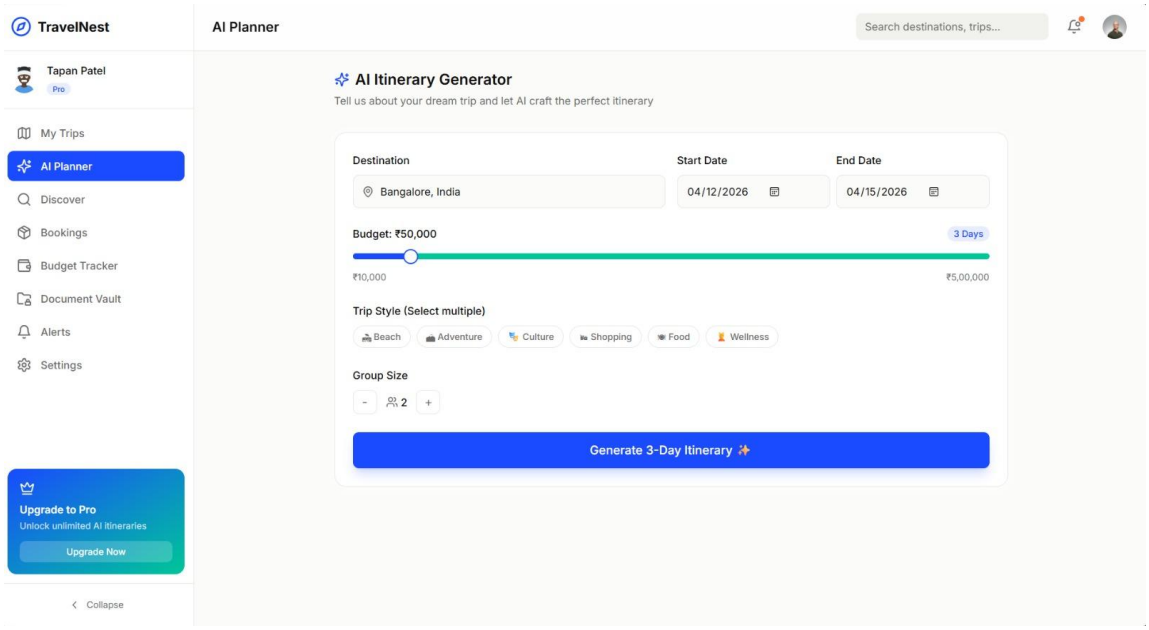


Figure 16: Trip Planner Interface

It shows the TravelNest Trip Planner module where users can create new trips by selecting a destination, setting travel dates, specifying group size and travel style, and defining a total budget. The interface displays existing trips with their current status indicators.

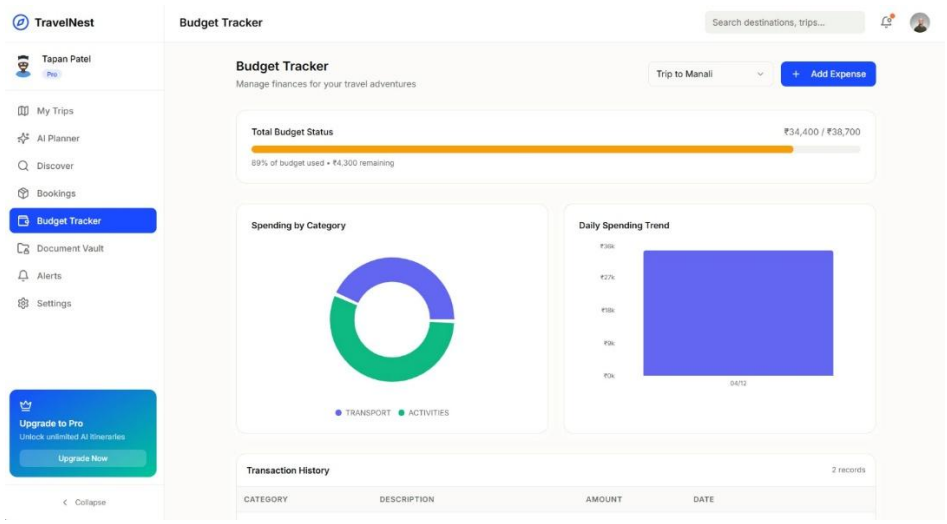


Figure 17: Budget Tracker Interface

This screenshot illustrates the Budget Tracker module, displaying a selected trip's total budget allocation alongside a categorized breakdown of recorded expenses. Visual progress bars indicate spending levels for each category

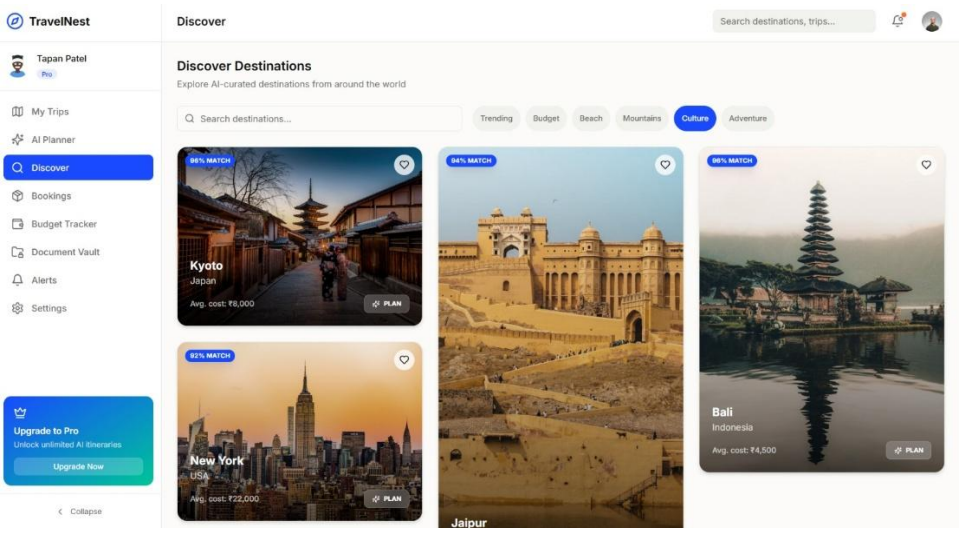


Figure 18: Destination Discovery Page

This screenshot presents the Destination Discovery module, showing a filterable grid of global travel destinations. Each destination card displays the destination name, country, average cost per day, community rating, and best travel seasons to assist users in trip selection.

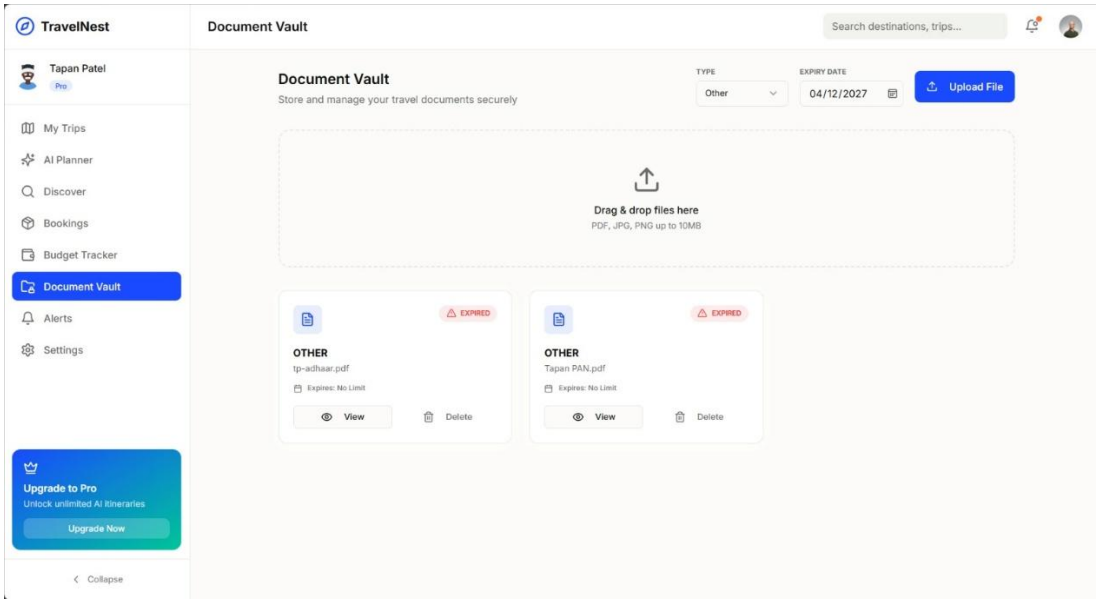


Figure 20: Document Vault Interface

This screenshot displays the Document Vault module where users can upload and manage travel documents such as passports, visas, and tickets.

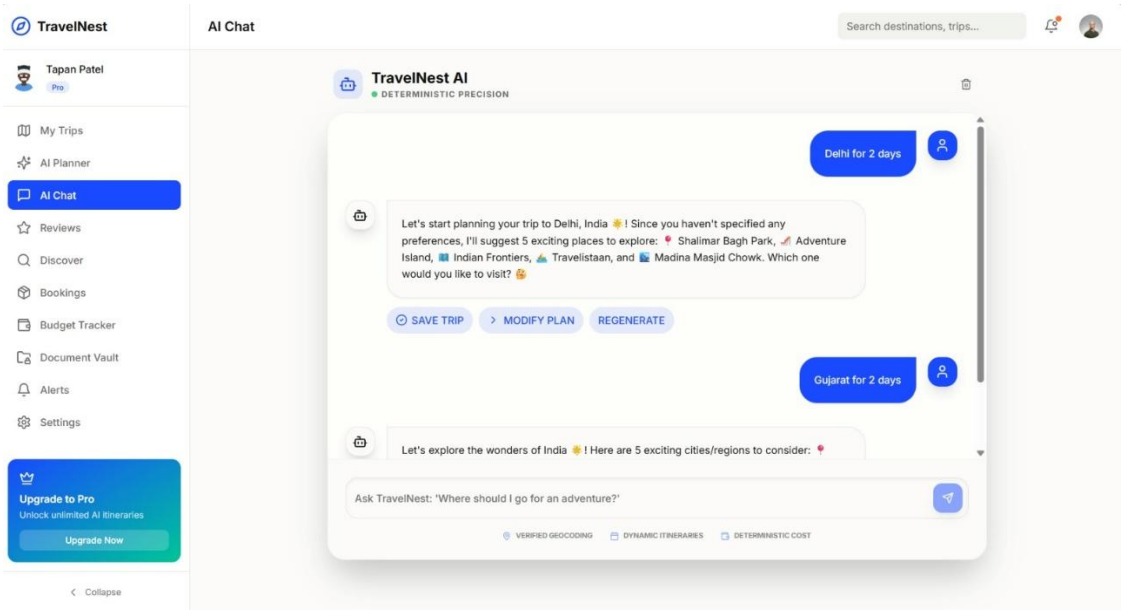


Figure 21: AI Trip Planner Interface

the AI Travel Planner interface where users can submit natural language travel queries EG:- A 2-day itinerary for a family trip to Delhi with a specific budget, and receive AI-generated recommendations.

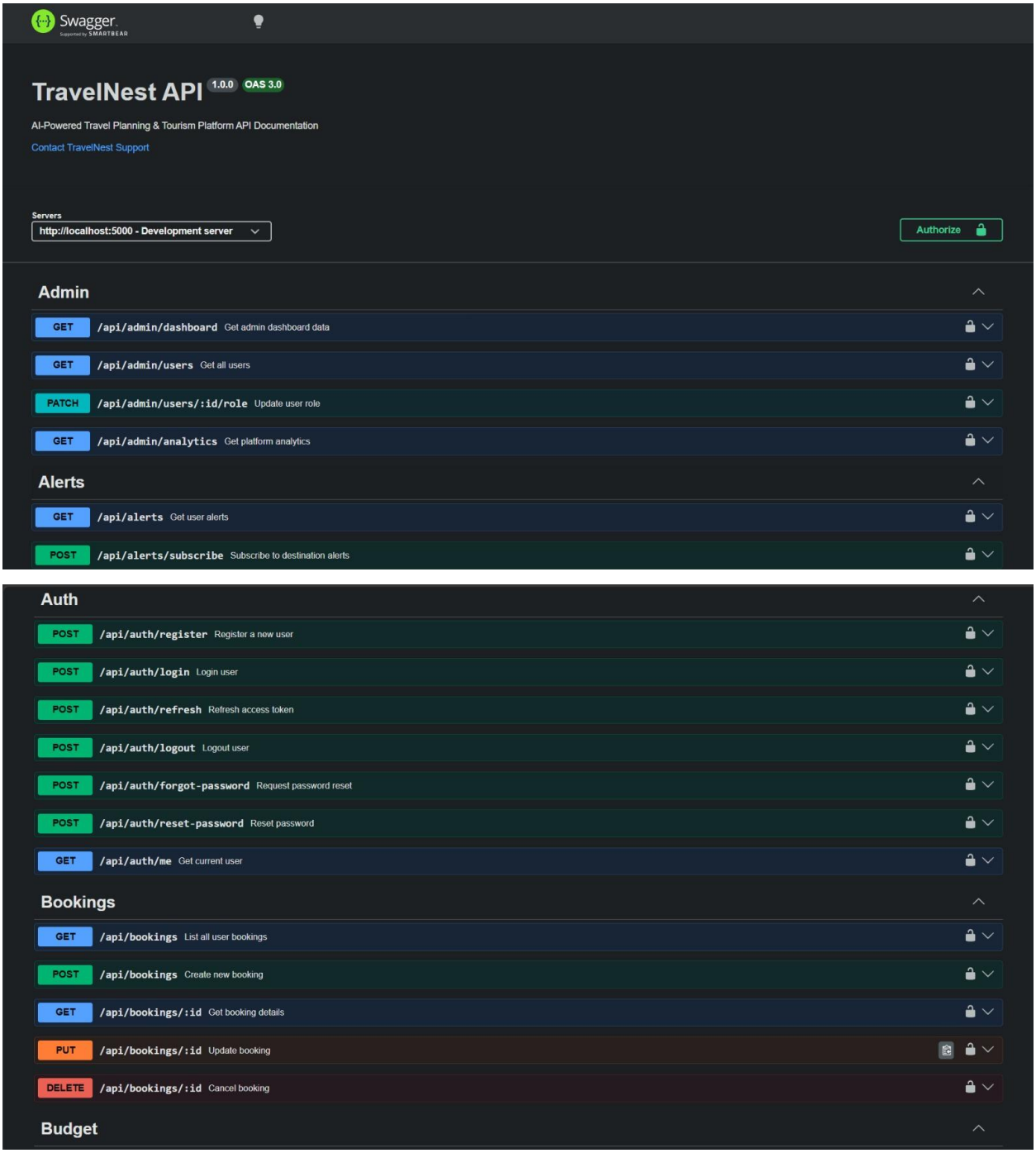


Figure 22: Swagger API Documentation

It displays the TravelNest Swagger UI, providing interactive documentation for all backend API endpoints. Each endpoint is documented with its HTTP method, request parameters, request body schema, and expected response formats, facilitating frontend-backend integration.

13. Individual Contribution of Team Members

The following table summarizes the specific contributions made by each team member in the design, development, and documentation of the TravelNest project:

Team Member	Roll Number	Key Contributions
Milav Patel	23BIT190	Frontend development using React and TypeScript. Designed and implemented the Landing Page, Authentication Pages, User Dashboard, Trip Planner, Destination Discovery, and Budget Tracker interfaces. Configured Tailwind CSS design system and Vite build pipeline.
Tapan Patel	23BIT158	Backend API development using Node.js, Express, and TypeScript. Implemented the Auth, Users, Trips, Destinations, and Bookings modules. Configured Prisma ORM with PostgreSQL schema, JWT authentication middleware, and Swagger API documentation.
Prem Patel	23BIT163	Docker containerization of all application services. Wrote Dockerfiles for the frontend, backend, and AI service containers. Configured the Docker Compose orchestration file with health checks, service dependencies, volume persistence, and Nginx reverse proxy routing.
Hozefa Travadi	23BIT165	Cloud infrastructure setup and AWS services integration. Configured the AWS S3 bucket for document storage, created AWS Lambda functions for notification processing, set up the AWS SQS alert queue, and integrated all AWS SDKs into the backend service. Documented the cloud architecture diagram.

All team members jointly participated in the problem identification, system architecture design, report preparation, and preparation for the evaluation viva and demonstration scheduled in April 2026. The project represents a collaborative academic effort, with each member contributing equitably to the overall quality of the TravelNest prototype.

References

1. Amazon Web Services. (2024). AWS Free Tier. https://aws.amazon.com/free/

2. Docker Inc. (2024). Docker Documentation. https://docs.docker.com/

3. Prisma. (2024). Prisma ORM Documentation. https://www.prisma.io/docs/

4. React. (2024). React Documentation. https://react.dev/

5. Node.js. (2024). Node.js Documentation. https://nodejs.org/en/docs/

6. Tailwind CSS. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs/

7. Vite. (2024). Vite Documentation. https://vitejs.dev/guide/

8. Redis. (2024). Redis Documentation. https://redis.io/docs/

9. PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation. https://www.postgresql.org/docs/15/

10. MongoDB, Inc. (2024). MongoDB Manual. https://www.mongodb.com/docs/manual/